

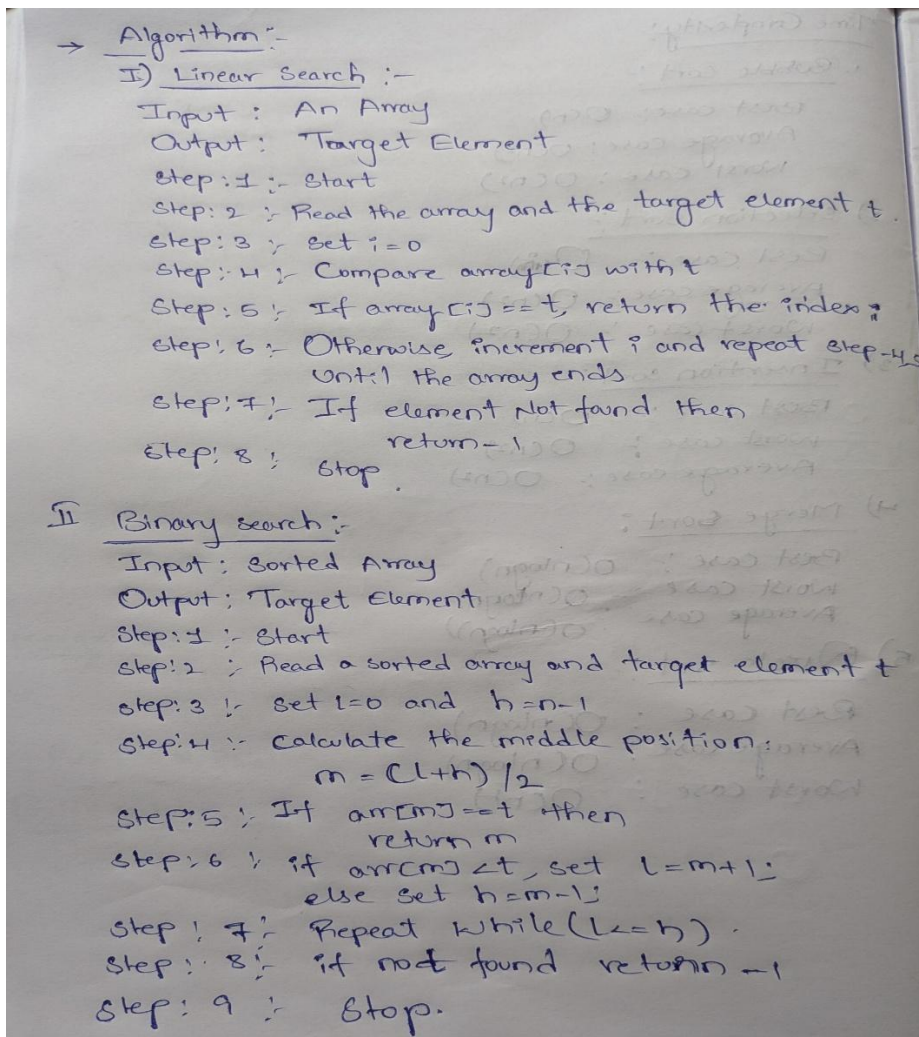
Practical: 2

Implementation and Time analysis of linear and binary search algorithm

05/08/2026

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Algorithm:



Program:

```
#include <iostream>
using namespace std;
```

```
// Linear Search
int linear(int array[], int s, int t) {
    for (int i = 0; i < s; i++) {
```

```
        if (array[i] == t) {
            return i;
        }
    }
    return -1;
}

// Binary Search
int binary(int array[], int s, int t) {
    int l = 0;
    int h = s - 1;

    while (l <= h) {
        int m = (l + h) / 2;

        if (array[m] == t) {
            return m;
        }
        else if (array[m] < t) {
            l = m + 1;
        }
        else {
            h = m - 1;
        }
    }

    return -1;
}

// Display Array
void display(int array[], int n) {
    cout << "Array is:\n";

    for (int i = 0; i < n; i++) {
        cout << array[i] << " ";
    }

    cout << endl;
}

int main() {

    int array[] = {2, 6, 7, 9, 11};
    int n = sizeof(array) / sizeof(array[0]);

    int t = 6;
```

```
display(array, n);

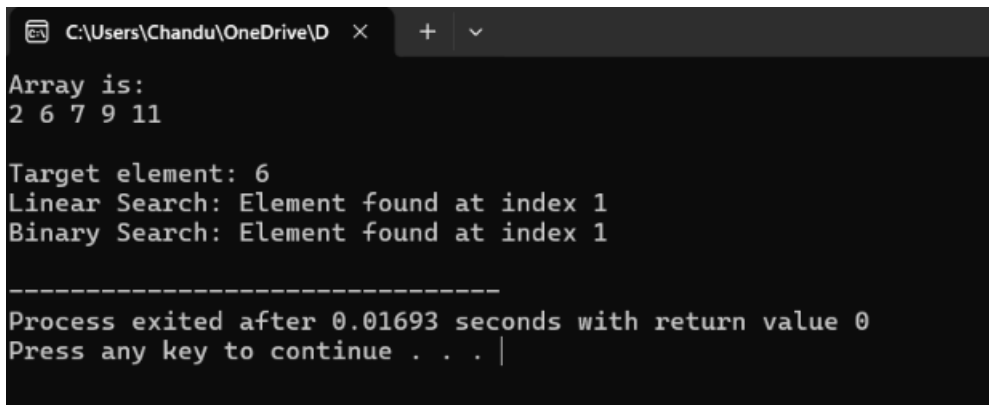
int f = linear(array, n, t);
int g = binary(array, n, t);

cout << "\nTarget element: " << t << endl;

if (f != -1) {
    cout << "Linear Search: Element found at index " << f << endl;
}
else {
    cout << "Linear Search: Element not found" << endl;
}

if (g != -1) {
    cout << "Binary Search: Element found at index " << g << endl;
}
else {
    cout << "Binary Search: Element not found" << endl;
}

return 0;
}
```

Output:

```
C:\Users\Chandu\OneDrive\D >
Array is:
2 6 7 9 11

Target element: 6
Linear Search: Element found at index 1
Binary Search: Element found at index 1

-----
Process exited after 0.01693 seconds with return value 0
Press any key to continue . . . |
```

Time Complexity:

Time Complexity :-

I) Linear Search :-

```

for (i=0; i<n; i++) {
    if (arr[i] == t) {
        return i;
    }
}
return -1;
    
```

$2n+2$
 n
 1
 1
 $2n+2 + n + 1 + 1$
 $3n+4$
 $T(n) = O(n)$

II) Binary Search :-

```

int binary(arr, s, t) {
    l = 0;
    h = s-1;
    while (l <= h) {
        m = (l+h)/2;
        if (arr[m] == t) {
            return m;
        }
        else if (arr[m] < t) {
            l = m+1;
        }
        else {
            h = m-1;
        }
    }
}
    
```

$n \cdot \frac{n}{2} + \frac{n}{2} + \frac{n}{2} + \dots$
 $\frac{n}{2^k} = 1$
 $n = 2^k$
 $k = \log n$
 $T(n) = O(\log n)$

Time Complexity :-

I) Linear Search :-

- Best case: $O(1)$
- Average case: $O(n)$
- Worst case: $O(n)$

II) Binary Search :-

- Best case: $O(1)$
- Average case: $O(\log n)$
- Worst case: $O(\log n)$

Conclusion:

Hence

From this practical of linear search, Binary search the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.